

SKILL WEEK6

Session – 11

TASK 1: To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency

```
GNU nano 8.7.1 session11_task1.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_HISTORY 10
#define MAX_COMMAND 200

int main()
{
    char history[MAX_HISTORY][MAX_COMMAND];
    char input[MAX_COMMAND];
    int count = 0;
    int next = 0;

    printf("Session 10 - Command History\n");
    printf("Enter commands (type 'history' to display, 'exit' to quit).\n\n");

    while (1)
    {
        printf("myshell> ");
        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0)
            continue;

        if (strcmp(input, "history") == 0)
        {
            printf("\nCommand History:\n");
            if (count == 0)
            {
                printf("History is empty.\n");
            }
            else
            {
                int start = (count < MAX_HISTORY) ? 0 : next;
                for (int i = 0; i < count; i++)
                {
                    int index = (start + i) % MAX_HISTORY;
                    printf("%d %s\n", i + 1, history[index]);
                }
            }
        }
    }
}
```

```

    }
    printf("\nHistory validation: SUCCESS\n");
}
continue;
}

if (strcmp(input, "exit") == 0)
{
    printf("\nExiting shell...\n");
    break;
}

strcpy(history[next], input);
next = (next + 1) % MAX_HISTORY;
if (count < MAX_HISTORY)
    count++;
printf("Command stored in history.\n");
}

printf("\nHistory buffer contains %d command(s).\n", count);
printf("History consistency verified.\n");
return 0;
}

```

```

rishill10@ASUS16X: ~/Week6_ × + ▾
rishill10@ASUS16X:~$ mkdir Week6_Skill
cd Week6_Skill
rishill10@ASUS16X:~/Week6_Skill$ nano session11_task1.c
rishill10@ASUS16X:~/Week6_Skill$ gcc session11_task1.c -o session11_task1
./session11_task1
Session 10 - Command History
Enter commands (type 'history' to display, 'exit' to quit).

myshell> ls
Command stored in history.
myshell> pwd
Command stored in history.
myshell> echo hello
Command stored in history.
myshell> histriy
Command stored in history.
myshell> exit

Exiting shell...

History buffer contains 4 command(s).
History consistency verified.
rishill10@ASUS16X:~/Week6_Skill$ |

```

TASK 2: To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations.

```
GNU nano 8.7.1 session11_task2.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_COMMANDS 10
#define MAX_ARGS 20
#define MAX_INPUT 500

typedef struct {
    char *args[MAX_ARGS];
    int argc;
} Command;

int parse_pipeline(char *input, Command commands[])
{
    int count = 0;
    char *part = strtok(input, "|");

    while (part != NULL && count < MAX_COMMANDS)
    {
        commands[count].argc = 0;
        char *arg = strtok(part, "\\t\\n");

        while (arg != NULL && commands[count].argc < MAX_ARGS - 1)
        {
            commands[count].args[commands[count].argc++] = arg;
            arg = strtok(NULL, "\\t\\n");
        }
        commands[count].args[commands[count].argc] = NULL;

        if (commands[count].argc == 0)
        {
            printf("Error: Empty command in pipeline.\\n");
            return -1;
        }
        count++;
        part = strtok(NULL, "|");
    }
}
```

```
    return count;
}

int main()
{
    char input[MAX_INPUT];
    Command commands[MAX_COMMANDS];

    printf("Session 10 - Pipeline Execution\\n");
    printf("Enter a pipeline command:\\n");
    printf("myshell> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    input[strcspn(input, "\\n")] = '\\0';

    if (strlen(input) == 0)
    {
        printf("Empty pipeline.\\n");
        return 0;
    }

    int command_count = parse_pipeline(input, commands);
    if (command_count <= 0)
    {
        printf("Pipeline validation failed.\\n");
        return 1;
    }

    printf("\\nPipeline Structure:\\n");
    for (int i = 0; i < command_count; i++)
    {
        printf("Command %d: ", i + 1);
        for (int j = 0; j < commands[i].argc; j++)
        {
            printf("%s ", commands[i].args[j]);
        }
        printf("\\n");
    }
}
```

```

printf("\nPipeline validation: SUCCESS\n");
printf("Execution order maintained.\n");

int pipes[MAX_COMMANDS - 1][2];
pid_t pids[MAX_COMMANDS];

for (int i = 0; i < command_count - 1; i++)
{
    if (pipe(pipes[i]) == -1)
    {
        perror("pipe failed");
        return 1;
    }
}

for (int i = 0; i < command_count; i++)
{
    pids[i] = fork();
    if (pids[i] < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pids[i] == 0)
    {
        if (i > 0)
        {
            dup2(pipes[i - 1][0], STDIN_FILENO);
        }
        if (i < command_count - 1)
        {
            dup2(pipes[i][1], STDOUT_FILENO);
        }
        for (int j = 0; j < command_count - 1; j++)
        {
            close(pipes[j][0]);
            close(pipes[j][1]);
        }

```

```

        execvp(commands[i].args[0], commands[i].args);
        perror("execvp failed");
        exit(EXIT_FAILURE);
    }
}

for (int i = 0; i < command_count - 1; i++)
{
    close(pipes[i][0]);
    close(pipes[i][1]);
}

for (int i = 0; i < command_count; i++)
{
    waitpid(pids[i], NULL, 0);
}

printf("All processes completed.\n");
printf("All pipe resources cleaned up.\n");
printf("Long pipeline executed successfully.\n");

return 0;
}

```

```
rishill10@ASUS16X: ~/Week6_ x + v
rishill10@ASUS16X:~/Week6_Skill$ nano session11_task2.c
rishill10@ASUS16X:~/Week6_Skill$ gcc session11_task2.c -o session11_task2
./session11_task2
Session 10 - Pipeline Execution
Enter a pipeline command:
myshell> ls | wc -l

Pipeline Structure:
Command 1: ls

Pipeline validation: SUCCESS
Execution order maintained.
session11_task1 session11_task1.c session11_task2 session11_task2.c
All processes completed.
All pipe resources cleaned up.
Long pipeline executed successfully.
rishill10@ASUS16X:~/Week6_Skill$ |
```

Session – 12

TASK – 1: To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow

```
GNU nano 8.7.1 session12_task1.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];
    pid_t child1, child2;

    if (pipe(pipefd) == -1)
    {
        perror("pipe failed");
        return 1;
    }

    printf("Pipe created successfully.\n");

    child1 = fork();
    if (child1 < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (child1 == 0)
    {
        close(pipefd[0]);
        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[1]);
        execlp("echo", "echo", "Hello from child process", NULL);
        perror("exec failed");
        exit(EXIT_FAILURE);
    }

    child2 = fork();
    if (child2 < 0)
    {
        perror("fork failed");
        return 1;
    }
}
```

```

    if (child2 == 0)
    {
        close(pipefd[1]);
        dup2(pipefd[0], STDIN_FILENO);
        close(pipefd[0]);
        execlp("wc", "wc", "-w", NULL);
        perror("exec failed");
        exit(EXIT_FAILURE);
    }

    close(pipefd[0]);
    close(pipefd[1]);

    printf("Connected commands executing...\n");

    waitpid(child1, NULL, 0);
    waitpid(child2, NULL, 0);

    printf("Data flow completed successfully.\n");
    printf("Parent process synchronized with both children.\n");

    return 0;
}

```

```

rishill10@ASUS16X: ~/Week6_ × + ▾
rishill10@ASUS16X:~/Week6_Skill$ nano session12_task1.c
rishill10@ASUS16X:~/Week6_Skill$ gcc session12_task1.c -o session12_task1
./session12_task1
Pipe created successfully.
Connected commands executing...
4
Data flow completed successfully.
Parent process synchronized with both children.
rishill10@ASUS16X:~/Week6_Skill$ |

```

TASK – 2: To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines.

```
GNU nano 8.7.1 session12_task2.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_COMMANDS 10
#define MAX_ARGS 20
#define MAX_INPUT 500

typedef struct {
    char *args[MAX_ARGS];
    int argc;
} Command;

int parse_pipeline(char *input, Command commands[])
{
    int count = 0;
    char *part = strtok(input, "|");

    while (part != NULL && count < MAX_COMMANDS)
    {
        commands[count].argc = 0;
        char *arg = strtok(part, " \\t\\n");

        while (arg != NULL && commands[count].argc < MAX_ARGS - 1)
        {
            commands[count].args[commands[count].argc++] = arg;
            arg = strtok(NULL, " \\t\\n");
        }
        commands[count].args[commands[count].argc] = NULL;

        if (commands[count].argc == 0)
        {
            printf("Error: Empty command in pipeline.\\n");
            return -1;
        }
        count++;
        part = strtok(NULL, "|");
    }
    return count;
}
```

```
}

int main()
{
    char input[MAX_INPUT];
    Command commands[MAX_COMMANDS];

    printf("Session 10 - Multiple Pipe Execution\\n");
    printf("Enter a long pipeline:\\n");
    printf("myshell> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    input[strcspn(input, "\\n")] = '\\0';

    if (strlen(input) == 0)
    {
        printf("Empty pipeline.\\n");
        return 0;
    }

    int command_count = parse_pipeline(input, commands);
    if (command_count <= 0)
    {
        printf("Pipeline validation failed.\\n");
        return 1;
    }

    printf("\\nPipeline Structure:\\n");
    for (int i = 0; i < command_count; i++)
    {
        printf("Command %d: ", i + 1);
        for (int j = 0; j < commands[i].argc; j++)
        {
            printf("%s ", commands[i].args[j]);
        }
        printf("\\n");
    }

    printf("\\nPipeline validation: SUCCESS\\n");
    printf("Execution order maintained.\\n");
}
```

```

int pipes[MAX_COMMANDS - 1][2];
pid_t pids[MAX_COMMANDS];

for (int i = 0; i < command_count - 1; i++)
{
    if (pipe(pipes[i]) == -1)
    {
        perror("pipe failed");
        return 1;
    }
}

for (int i = 0; i < command_count; i++)
{
    pids[i] = fork();
    if (pids[i] < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pids[i] == 0)
    {
        if (i > 0)
        {
            dup2(pipes[i - 1][0], STDIN_FILENO);
        }
        if (i < command_count - 1)
        {
            dup2(pipes[i][1], STDOUT_FILENO);
        }
        for (int j = 0; j < command_count - 1; j++)
        {
            close(pipes[j][0]);
            close(pipes[j][1]);
        }

        execvp(commands[i].args[0], commands[i].args);
        perror("execvp failed");
        exit(EXIT_FAILURE);
    }
}

```

```

    }
}

for (int i = 0; i < command_count - 1; i++)
{
    close(pipes[i][0]);
    close(pipes[i][1]);
}

for (int i = 0; i < command_count; i++)
{
    waitpid(pids[i], NULL, 0);
}

printf("All processes completed.\n");
printf("All pipe resources cleaned up.\n");
printf("Long pipeline executed successfully.\n");

return 0;
}

```

```
rishill10@ASUS16X: ~/Week6_ × + ∨  
rishill10@ASUS16X:~/Week6_Skill$ nano session12_task2.c  
rishill10@ASUS16X:~/Week6_Skill$ gcc session12_task2.c -o session12_task2  
./session12_task2  
Session 10 - Multiple Pipe Execution  
Enter a long pipeline:  
myshell> ls | grep c | wc -l  
  
Pipeline Structure:  
Command 1: ls  
  
Pipeline validation: SUCCESS  
Execution order maintained.  
session11_task1    session11_task2    session12_task1    session12_task2  
session11_task1.c  session11_task2.c  session12_task1.c  session12_task2.c  
All processes completed.  
All pipe resources cleaned up.  
Long pipeline executed successfully.  
rishill10@ASUS16X:~/Week6_Skill$ |
```